

Experimental and Modeling Skills

- **Experimental skills** involve the ability to analyze the real world problems.
- Focus on designing, conducting, and interpreting results from physical or simulated experiments.
- Involve:
 - Research design (developing a plan, methodology)
 - Selection of variables for experiments
 - Data collection
 - Conduct experiment
 - Data analysis
 - Result evaluation and validation
 - Dissertation

Experimental and Modeling Skills

- **Modeling skills** involve creating simplified representations (models) of systems to predict behavior or understand complex phenomena.
- Include
 - Model formulation
 - Analysis
 - Interpretation
 - Validation
 - Improvement/feedback
- Experimental skills and Modeling skills work together to enhance the understanding of the real world problems, and provide solutions.

Selection of Variables

- Independent variables (factor):
 - The variables whose values are changed to see how they affect the result/outcome.
 - Also called input variables.
- Dependent variables (response):
 - Affected by the changes in the independent variables.
 - Also called output variables.
- Controlled variables (constant):
 - The variables that are kept unchanged throughout the experiment.
 - No effect on the result/outcome.
- Noise Variables:
 - Uncontrolled variables that results variation in the result.
 - Creates random effects in the model.

Selection of Variables

- Example: “*A Survey on Programming Language Preferences and Their Impact on a Developer’s Productivity*”.
 - Possible independent variables:
 - ✓ Preferred programming language (Java, Python, PHP)
 - ✓ Frequency of language used.
 - ✓ Types of the developer (web, software, database, system, data science, game, mobile, AI, ML)
 - ✓ Programming experience (in years)
 - ✓ IDE or tools used
 - ✓ Operating system (Windows, Linux, MacOS)
 - These variables influence the productivity of a developer.

Selection of Variables

- Possible dependent variables:
 - ✓ Developer's productivity score
- Possible controlled variables:
 - ✓ Levels of a developer (junior, senior, expert)
 - ✓ Type of the project
 - ✓ Working hours/day
- Possible noise variables:
 - ✓ Workplace environment
 - ✓ Developer's mental state
 - ✓ Personal biasness in language

Design Matrix

➤ Here, each combination is tested against multiple developers.

Run	Programming Language	Experience Level	Productivity Score (S)
1	Java	4 years	S ₁
2	Python	4 years	S ₂
3	Java	7 years	S ₃
4	Python	7 years	S ₄

2-level Factorial Design

- It is a structured experimental design used to study the effects of two or more independent variables, each at **two levels** (high/low, yes/no, Java/Python), on a dependent variable (productivity score).
- Example:
 - ✓ **Step 1:** Select the independent variables:

Factor	Level 0	Level 1
A: Preferred programming language	Java	Python
B: Programming experience	4 years	7 years

2-level Factorial Design

✓ Step 2: 2-level Factorial Design Matrix ($2^2 = 4$ Runs)

Run	A: Preferred programming language	B: Programming experience	Coded Values	Description
1	Java	4 years	A = 0, B = 0	Baseline
2	Python	4 years	A = 1, B = 0	Test language effect (Junior)
3	Java	7 years	A = 0, B = 1	Test experience effect (Java)
4	Python	7 years	A = 1, B = 1	Test combined effect

2-level Factorial Design

✓ **Step 3:** Suppose data is collected from 3 developers for each combination.

Run	Language	Experience	Score of Developer 1	Score of Developer 2	Score of Developer 3	Average
1	Java	4 years	69	70	68	69
2	Python	4 years	73	72	72	72.33
3	Java	7 years	80	82	83	81.66
4	Python	7 years	86	88	87	87

2-level Factorial Design

✓ Step 4: Calculate effect:

- E_1 : Java, 4 years
 E_2 : Python, 4 years
 E_3 : Java, 7 years
 E_4 : Python, 7 years

- Now calculate: Main effect of A (Preferred Programming Language) = $\frac{(E_2 + E_4) - (E_1 + E_3)}{2}$

$$\text{Main effect of B (Programming Experience)} = \frac{(E_3 + E_4) - (E_1 + E_2)}{2}$$

$$\text{Interaction (AB)} = \frac{(E_1 + E_4) - (E_2 + E_3)}{2}$$

- Analysis can be done in Excel. Other statistical tools can be used also.

2-level Factorial Design

- Based on Step 3, the calculated effects are:
 - Main effect of A (Language) = $\frac{(E_2 + E_4) - (E_1 + E_3)}{2} = \frac{(72.33 + 87) - (69 + 81.66)}{2} = 4.33$
 - Main effect of B (Experience) = $\frac{(E_3 + E_4) - (E_1 + E_2)}{2} = \frac{(81.66 + 87) - (69 + 72.33)}{2} = 27.33$
 - Interaction (AB) = $\frac{(E_1 + E_4) - (E_2 + E_3)}{2} = \frac{(69 + 87) - (72.33 + 81.66)}{2} = 1.00$

Outcome of the Experimental Analysis

- After analyzing the experiment of developer's productivity, the key findings are:
 - On an average, Python developers' productivity is increase by 4 points.
 - Experienced developers (7 years) are more productive by 27 points.
 - Mixed combination results minimal impact; insignificant.
- So, finally it can be said that, experience plays a strong/major role regardless of language and Python developers proof more productivity. The combination of tasks (language and experience) does not improve the productivity of a developer much.

3-level Factorial Design

➤ Do yourself.

Fractional Factorial Design

- A **Fractional Factorial Design** uses only a **fraction** of the total combinations in a full factorial design, while still giving useful insights about **main effects** and some **interactions**.
- It is done when,
 - Testing many factors.
 - Time/resources are limited.